

AIMLCZG546 · SESSION 4 · COMPANION READER

Quality Attributes and System Architecture

From Non-Functional Requirements to Architectural Patterns for ML Systems

Session 4 · Read alongside the lecture slides · Every concept and case study explained in full

How to use this companion. **Blue** = intuition. **Amber** = everyday analogy. **Green** = worked scenario. **Red** = pitfall. **Purple** = one line to remember.

1 Case Studies: Flipkart Failure vs Hotstar Success

Two dramatic data points establish why architecture matters before a single line of code.

1.1 Flipkart Big Billion Day 2014 (Failure)

Flipkart announced India's biggest sale. Three lakh (300,000) orders arrived in six hours. The website crashed. Cart items vanished. Money was deducted but orders were not placed. The root cause: Flipkart had not identified their key quality attributes and had not built an architecture capable of meeting them.

1.2 Hotstar Cricket World Cup 2019 (Success)

India vs New Zealand semi-final. **25 million concurrent users** watched on Hotstar with zero crashes. How? Hotstar's engineering team had explicitly identified scalability, availability, and performance as their key quality attributes years before, and every architectural decision — microservices, CDN edge caching, adaptive bitrate streaming, auto-scaling Kubernetes clusters — was made to satisfy those attributes.

Worked example: Comparing the two architectures

Flipkart (2014): Monolithic application, single database, no auto-scaling. Quality attributes never formalised.

When 300,000 users arrived simultaneously, the single database became the bottleneck. Sessions were stored in memory on individual servers. When a server restarted under load, all sessions (and cart contents) were lost. There was no fallback.

Hotstar (2019): Microservices, geographically distributed CDN, stateless video delivery, auto-scaling groups. Scalability QA: "handle 25M concurrent streams at ≤ 2 s buffer time." Availability QA: "99.99% uptime during live events."

Every infrastructure decision was tested against these two numbers in load tests months before the World Cup.

✓ Key takeaway

Quality attributes are not an afterthought. They must be specified before architecture is chosen. The architecture exists to satisfy the QAs.

2 What Are Quality Attributes?

Quality Attributes (QAs), also called **Non-Functional Requirements (NFRs)**, specify *how well* a system does what it does, not *what* it does.

Formal definition (Len Bass): “A *quality attribute* is a measurable or testable property of a system that is used to indicate how well the system satisfies the needs of its stakeholders.”

The intuition

Functional requirements are the *what*: “The system allows users to place orders.” Quality attributes are the *how well*: “The system allows users to place orders in under 2 seconds, 99.9% of the time, even with 1 million simultaneous users.” The second version is engineering-ready; the first is not.

2.1 SMART Quality Attributes

Every QA must be SMART:

- **Specific** — clearly defined, not vague (“fast” is not specific; “ $\leq 200\text{ms}$ latency” is)
- **Measurable** — can be quantified and tested
- **Attainable** — realistic given technology and budget
- **Relevant** — directly tied to a business or user need
- **Time-sensitive** — measured under a specific load or condition

Worked example: QAs for a food delivery app (Swiggy/Zomato)

QA	Vague version	SMART version
Performance	Fast response	Load restaurant menus in $\leq 2\text{s}$; order placement in $\leq 3\text{s}$ at P95
Availability	Always online	99.95% uptime annually; $24 \times 7 \times 365$
Usability	Easy to use	90% of new users place first order without assistance
Scalability	Handle peak traffic	10M orders/second at peak with $\leq 5\text{s}$ degradation
Interoperability	Works with payments	99% successful transactions with Razorpay and Google Maps APIs
Reliability	Correct orders	99.9% order accuracy (right food to right address)

3 Quality Attributes Specific to ML Systems

ML systems have additional QAs not found in traditional software. These directly affect model selection, pipeline design, and infrastructure choices.

ML QA	Definition	Example target
Accuracy	How correctly model predicts	$F1 \geq 0.88$ on held-out test set
Scalability	Handle growing data/traffic	Serve 10,000 predictions/sec without latency spike
Reliability	Consistent performance across conditions	P99 latency ≤ 500 ms under $5\times$ normal load
Explainability	Humans can understand model decisions	SHAP explanation generated for every prediction
Robustness	Maintains accuracy on noisy/adversarial input	Accuracy drops $\leq 3\%$ when 10 % of input features are missing
Maintainability	Models/pipelines can be updated easily	Any model update deployable in ≤ 1 hour with zero downtime
Security & Privacy	Protected from attacks and data leaks	PII never stored in logs; models resist adversarial perturbations
Fairness	Unbiased outcomes across demographics	Recall gap $\leq 2\%$ across gender groups
Data Drift Adaptability	Detect and adapt to input distribution shift	Drift alert triggered when KL divergence > 0.1 ; retrain within 24h
Model Drift Monitoring	Track performance degradation post-deployment	Weekly evaluation on labelled production sample
Reproducibility	Same training consistently produces same results	Given same data + seed, model weights differ by < 0.001

★ Everyday picture

Robustness is like a car that still steers correctly in rain, fog, and snow — not just on a sunny test track. A fraud model must work when some transaction fields are missing (network timeout), when amounts are in an unusual currency, and when a new type of attack appears.

Where this shows up in ML/AI. Every ML QA directly drives an architectural choice. Explainability $\Rightarrow$ choose an interpretable model or add a SHAP layer. Reproducibility $\Rightarrow$ fix random seeds, version datasets, use deterministic training pipelines. Fairness $\Rightarrow$ stratified sampling, fairness-aware loss functions, bias audits.

4 System Architecture

Software architecture (IEEE definition): “*The software architecture of a program or computing system is the structure or structures of the system, which comprise software elements, the externally visible properties of those elements, and the relationships among them.*”

A full **system architecture** includes five layers working together:

1. **Hardware** — servers, GPUs, storage devices, network equipment.
2. **Infrastructure** — cloud services (AWS/Azure/GCP), containers (Docker), orchestration (Kubernetes).
3. **Software** — APIs, authentication, business logic services.
4. **ML Components** — feature store, model registry, serving engine, monitoring.
5. **Data** — vector database, relational database, object storage, streaming pipelines.

Worked example: RAG enterprise chatbot architecture

A company builds an internal HR policy chatbot. The system architecture:

Hardware/Infra: AWS EC2 GPU instances for embedding generation; ECS Fargate for the API.

Software: API Gateway (rate limiting, auth) $\rightarrow$ Orchestration Layer (LangChain).

ML Components: Embedding Model (sentence-transformers/all-MiniLM) $\rightarrow$ Vector DB (Pinecone) $\rightarrow$ LLM (GPT-4 via API) $\rightarrow$ Re-ranking Model.

Data: HR policy documents in S3 $\rightarrow$ chunked and indexed in Pinecone; interaction logs in PostgreSQL.

Deployment: Docker containers, GitHub Actions CI/CD, CloudWatch monitoring.

The architecture was chosen to satisfy: Accuracy (GPT-4), Explainability (source docu-

ments cited in every answer), Maintainability (new policies added to S3 trigger automatic re-indexing), and Security (no PII leaves the company network).

5 Architectural Patterns

An **architectural pattern** is a general, reusable solution to a commonly occurring software architecture problem within a given context. Expressed as a triple: {Context, Problem, Solution}.

The intuition

A pattern is a proven template, not a finished design. Just as a house architect chooses from known patterns (open-plan kitchen-living, split-level, courtyard) that have worked in similar climates and family sizes, a software architect chooses from known patterns that have worked in similar system contexts.

Common patterns in ML systems:

- **Pipe-and-Filter** — sequential processing stages (this session)
- **CQRS** — separate command and query paths (Session 5)
- **Event-Driven** — publish-subscribe messaging (Session 6)
- **Monolithic** — single deployable unit (Session 5)
- **Microservices** — independent services (Session 5)

6 Pipe-and-Filter Pattern

Context: A system must process a stream of data through a sequence of transformations. Each transformation is independent and could be reused or replaced.

Problem: How do you organise the system so that each processing step can be developed, tested, and replaced independently?

Solution: The **Pipe-and-Filter** pattern. Each processing unit (**filter**) receives data from a **pipe**, transforms it, and passes the result to the next pipe. Filters are independent — they do not communicate with each other directly.

The intuition

Think of an assembly line in a factory. Station 1 cuts the metal. Station 2 welds it. Station 3 paints it. Station 4 inspects it. Each station only knows about its own job. You can add a polishing station between paint and inspection without changing any other station. That modularity is the core benefit of Pipe-and-Filter.

Worked example: NLP sentiment analysis pipeline

Input sentence: "I am not happy with the service!"

Filter 1 — Tokenise:

```
["I", "am", "not", "happy", "with", "the", "service", "!"]
```

Filter 2 — Remove stopwords:

```
["not", "happy", "service"]
```

Filter 3 — Vectorise (bag-of-words):

```
{"not":1, "happy":1, "service":1}
```

Filter 4 — Model inference (logistic regression):

$$\hat{y} = \sigma(w_0 + w_{\text{not}} \cdot 1 + w_{\text{happy}} \cdot 1 + w_{\text{service}} \cdot 1) = 0.89$$

Class = NEGATIVE (threshold 0.5).

Filter 5 — Format output:

```
{"text": "I am not happy with the service!",  
 "sentiment": "NEGATIVE", "confidence": 0.89}
```

Each filter can be swapped independently. Replacing Filter 3 with TF-IDF or Filter 4 with BERT requires no changes to any other filter.

✗ Watch out

Pipe-and-Filter assumes each filter is stateless. If a filter must remember state across multiple inputs (e.g., a streaming model that learns online), the pattern must be modified. Stateful filters break the independence guarantee.

✓ Key takeaway

Pipe-and-Filter makes ML pipelines modular. Each step (preprocessing, feature extraction, inference, postprocessing) can be tested, versioned, and replaced independently.